

RELATÓRIO DO PROJETO - SIMULADOR DE SOBREVIVÊNCIA

Participantes: Lázaro Fornari da Silva e Kauã Gabriel Lorenssetti
Ano: 2026

1. Introdução

O projeto consiste em um jogo de sobrevivência e aventura desenvolvido para funcionar diretamente no terminal. Desde o início, nossa intenção foi criar uma experiência fácil de compreender, mas que não se resumisse a apenas escolher opções em um menu. Por isso, incluímos diferentes caminhos, combates e uma progressão que depende das decisões tomadas ao longo da partida.

No jogo, o jogador cria seu personagem, escolhe uma classe e começa a explorar o mapa. Durante a jornada, enfrenta inimigos, conquista ouro e encontra itens que podem ajudá-lo nas próximas batalhas. Para avançar até o desafio final, que é derrotar o dragão, também precisa administrar a vida do personagem, o inventário e os equipamentos disponíveis.

2. Etapas do desenvolvimento

Antes de começar a programar, pensamos no caminho que o jogador percorreria do início ao fim. A partir disso, dividimos o jogo em etapas: menu principal, criação do personagem, escolha da classe, acesso ao mapa, exploração e combate. Também definimos quais informações precisariam continuar disponíveis durante toda a execução, como nome, vida, dano, agilidade, inventário, ouro e equipamentos.

Com essa estrutura organizada, desenvolvemos os menus e as funções responsáveis pelas principais ações do jogo. Depois que a base estava funcionando, acrescentamos a loja, os baús, as melhorias de equipamentos e os acontecimentos aleatórios. Por fim, conectamos todas essas partes e testamos um percurso completo, começando pela criação do personagem, passando por uma batalha e terminando com o retorno ao mapa.

3. Funcionamento do jogo

A partida começa com a criação do personagem. O jogador pode escolher entre três classes: guerreiro, arqueiro e escudeiro. Cada uma possui valores próprios de vida, dano e agilidade, o que muda um pouco a forma de jogar. Todos começam com uma poção de cura e, depois dessa escolha, o mapa é apresentado com quatro destinos possíveis: floresta, vilarejo, castelo e caverna do dragão.

Cada lugar possui um inimigo específico. Os combates acontecem em turnos, e o jogador pode atacar, recuar ou abrir o inventário. Durante o ataque, existe a chance de ocorrer um golpe crítico; na vez do inimigo, a agilidade do personagem pode permitir que ele desvie. A batalha termina quando a vida de um dos dois chega a zero. Ao vencer, o jogador recebe ouro e ainda pode encontrar uma loja ou um baú pelo caminho. Toda essa preparação leva ao principal objetivo da aventura: chegar à caverna e derrotar o dragão.

4. Estruturas e recursos utilizados

As classes Personagem e Inimigo foram criadas para reunir os atributos de cada entidade e facilitar a atualização desses valores durante a partida. As listas armazenam as classes disponíveis, os itens do inventário e as possíveis recompensas dos baús. Já a divisão do código em funções e métodos ajudou a separar as tarefas de explorar, batalhar, atacar, comprar e usar itens, deixando o programa mais organizado.

Também utilizamos laços de repetição para manter os menus ativos e estruturas condicionais para conferir as escolhas do jogador. O módulo random foi responsável pelos elementos de sorte, como golpes críticos, desvios, encontros e recompensas. O módulo time, por sua vez, permitiu inserir pequenas pausas entre as mensagens, tornando a leitura no terminal mais agradável.

5. Dificuldades encontradas e soluções

Uma das primeiras dificuldades foi evitar que uma resposta inválida encerrasse o jogo. Para resolver isso, aplicamos tratamento de erros nas escolhas numéricas e verificamos, em cada menu, se a opção digitada realmente existia. Outro desafio foi manter os dados do personagem atualizados em todas as partes do programa. A solução encontrada foi concentrar essas informações no próprio objeto Personagem e passar esse mesmo objeto para as funções do mapa, da exploração e da batalha.

Também precisamos lidar com situações que poderiam causar problemas durante a partida, como vida abaixo de zero, inventário cheio, falta de ouro ou tentativa de equipar dois itens do mesmo tipo. Antes de concluir cada ação, o programa realiza as verificações necessárias. Além disso, usamos probabilidades em ataques, desvios, encontros com a loja e itens dos baús, evitando que todas as partidas aconteçam exatamente da mesma maneira.

6. Testes realizados

Depois de concluir o desenvolvimento, verificamos a sintaxe do arquivo e executamos o jogo pelo terminal. Entre os testes realizados estiveram a entrada e a saída do menu, a criação de um guerreiro, a exploração da floresta, a batalha contra o lobo, o recebimento de ouro, a caminhada, a decisão de abrir ou não um baú e o retorno ao mapa. Esse percurso foi concluído sem erros. Como parte dos acontecimentos depende de sorte, outras partidas podem apresentar resultados diferentes, o que já era esperado.

7. Conclusão

Ao final do projeto, conseguimos desenvolver um jogo de terminal com criação de personagem, exploração, combate, inventário, loja, equipamentos e evolução ao longo da partida. A organização do programa em funções, métodos e classes tornou o código mais claro e facilitou tanto os testes quanto a correção de problemas.

O resultado alcançou a proposta inicial e ainda deixa espaço para novas melhorias. Em versões futuras, seria possível incluir um sistema para salvar a partida, coleta de recursos, criação de itens e novos acontecimentos aleatórios, ampliando as escolhas e tornando cada aventura ainda mais completa.

8. Fontes consultadas

PYTHON SOFTWARE FOUNDATION. Estruturas de dados. Disponível em: <https://docs.python.org/pt-br/3/tutorial/datastructures.html>. Acesso em: 18 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Classes. Disponível em: <https://docs.python.org/pt-br/3/tutorial/classes.html>. Acesso em: 18 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Módulo random. Disponível em: <https://docs.python.org/pt-br/3/library/random.html>. Acesso em: 18 ago. 2026.

9. Declaração sobre o uso de inteligência artificial

A inteligência artificial foi utilizada como ferramenta de apoio na revisão, nos testes do arquivo enviado e na organização deste relatório. Nesta etapa, o código-fonte do projeto não foi alterado pela inteligência artificial.